Representation Effects of the EML Operator Under Real-Domain Constraints: A Structural Analysis

Biswajyoti Nath
Independent Study
github.com/biswajyoti-nath/eml-framework

May 2026

Abstract

The EML operator, $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$, was recently shown to generate the full repertoire of elementary functions when composed with the constant 1 [1]. This theoretical universality, however, holds over the complex field under conditions that are unavailable in constrained real-valued computation. We study how enforcing a single nonlinear primitive changes the symbolic structure of expressions under such constraints. Concretely, we implement a domain-safe recursive rewrite system that transforms the supported exp–log fragment into EML form, and measure the structural consequences across a 23-expression benchmark suite. Our analysis reveals a consistent pattern: the EML transformation homogenises the nonlinear layer of expression trees into a single operator type, but this structural regularity incurs a representational overhead in the form of increased node count and tree depth. Domain constraints—in particular the strict positivity requirements imposed by real-valued logarithms—are shown to be a primary shaping force on the resulting representations, not a peripheral inconvenience. An exploratory symbolic regression experiment further illustrates how this altered representation modifies the topology of the search space. This paper is a representation study; no claims of computational superiority or universality are made.

1 Introduction

Elementary functions—exponentials, logarithms, powers, and their compositions—form the computational substrate of mathematical modelling across science and engineering. Their evaluation has historically required a heterogeneous set of operations, each implemented separately in hardware and software. A natural question is whether this diversity is essential, or whether a single universal primitive could subsume it.

A striking answer was recently provided by Odrzywołek [1], who proved that the single binary operator

$$\mathrm{eml}(x, y) \;=\; \exp(x) - \ln(y), \tag{1}$$

together with the constant 1, suffices to express every elementary function. Under the grammar $S \to 1 \mid \mathrm{eml}(S, S)$, every such function becomes a uniform binary tree of identical nodes. The paper further demonstrates differentiable symbolic regression over EML trees as parameterised circuits.

The theoretical result is elegant and constructive, but it operates over the complex field and is silent on computational efficiency or domain safety. In practical real-valued computation, the gap between theory and deployment is significant: logarithm arguments must be strictly positive, non-integer powers require positive bases, and the complex-valued identities that underpin certain EML encodings are simply unavailable. How, then, does the EML operator behave when these idealisations are removed?

This paper addresses that question directly. We study how enforcing a single nonlinear primitive changes symbolic structure under realistic domain constraints. Rather than treating EML as an encoding trick, we treat it as a *representation transformation*—a rewriting that alters the shape, depth, and operator composition of expression trees in ways that carry computational implications. Our contributions are:

1. A recursive, domain-enforcing symbolic rewriter that transforms the supported real-valued exp–log fragment into EML form, raising explicit exceptions for unsupported expressions and domain violations.
2. A systematic structural comparison of EML and standard exp/log representations across a 23-expression benchmark suite, measuring nonlinear node count, tree depth, unique subexpressions, and weighted circuit cost.
3. An exploratory evolutionary symbolic regression experiment that examines how the EML grammar shapes search-space behaviour relative to a standard exp/log grammar.

This study does not extend, contradict, or claim to validate the theoretical result of Odrzywołek [1]. It analyses one constrained instantiation of that operator in a practical computational context.

2 Preliminaries

Definition 1 (EML operator)**.** *The EML (Exp-Minus-Log) operator is the binary function* $\mathrm{eml} : \mathbb{R} \times \mathbb{R}_{>0} \to \mathbb{R}$ *defined by*

$$\mathrm{eml}(x, y) = \exp(x) - \ln(y). \tag{2}$$

The second argument must be strictly positive for the result to be real-valued.

2.1 Induced Representations

The primitive directly encodes two fundamental nonlinear operations:

$$\exp(x) = \mathrm{eml}(x, 1), \tag{3}$$
$$\ln(x) = 1 - \mathrm{eml}(0, x), \quad x > 0. \tag{4}$$

Under positive-domain assumptions, the exp–log identities extend to arithmetic operations:

$$a \cdot b = \exp\big(\ln(a) + \ln(b)\big), \quad a, b > 0, \tag{5}$$
$$a/b = \exp\big(\ln(a) - \ln(b)\big), \quad a, b > 0, \tag{6}$$
$$x^n = \mathrm{sign}(x)^n \cdot \exp\big(n \cdot \ln|x|\big), \quad n \in \mathbb{Z} \setminus \{0\}, \tag{7}$$
$$x^a = \exp\big(a \cdot \ln(x)\big), \quad x > 0,\ a \notin \mathbb{Z}. \tag{8}$$

Remark 1. *Multiplication can only be absorbed into the EML primitive when both operands are provably positive. Addition is not expressible within the primitive at all; it acts as an external combinator throughout our fragment. These two facts are not implementation limitations—they reflect the boundary of what the positive-domain fragment can represent.*

2.2 Supported Fragment

We define the *supported positive-domain exp–log fragment* as the set of expressions built from exp, ln, $|\cdot|$, addition, multiplication, integer and rational Pow, and derivatives (evaluated before transformation). All other functions—including trigonometric and hyperbolic functions—are explicitly rejected.

3 Transformation Method

The core of our system is a recursive rewriter that maps each node of a SymPy expression tree to its EML representation, subject to domain constraints enforced at transform time.

3.1 Structural Rewrite Rules

The rewriter dispatches on the node type of the input expression. Its structure reflects the representational boundary described in Section 2: the linear layer (addition) passes through unchanged, while the nonlinear layer is fully absorbed into eml primitives.

Listing 1: Recursive transformation dispatch.

```
def transform(expr):
    if expr.is_Atom:     return expr
    if expr.is_Derivative: return transform(expr.doit())
    if expr.is_Add:      return Add(*map(transform, expr.args))
    if expr.is_Mul:      return transform_mul(expr) # domain-checked
    if expr.is_Pow:      return transform_pow(expr) # domain-checked
    if expr.func == exp: return eml(transform(expr.args[0]), S.One)
    if expr.func == log: return S.One - eml(S.Zero,
                                transform(expr.args[0]))
    raise UnsupportedExpressionError(expr)
```

The structure of this dispatch encodes a representational choice: the addition branch is the only point at which heterogeneous operators persist in the output tree. Every other supported node type is rewritten into the uniform EML form.

3.2 Domain Enforcement

Three conditions are checked at transform time and cannot be bypassed:

1. **Logarithm positivity.** `transform_mul` and the non-integer case of `transform_pow` require that ln arguments be provably positive under SymPy's assumption system. Failure raises `DomainError`; there is no silent fallback.

2. **Multiplicative domain.** If any factor in a product is not provably positive, the exp–log absorption (5) cannot be applied and a `DomainError` is raised immediately.

3. **Non-integer power bases.** Fractional exponentiation via (8) requires a positive base; otherwise a `NotImplementedError` is raised.

These checks are conservative: a symbol without an explicit positivity assumption in SymPy is treated as potentially negative. Expressions that are safe on a user-specified domain but involve unconstrained symbols will therefore be rejected symbolically, and may require numeric validation as a secondary check.

This rigidity is a design choice, not a deficiency. Silent domain violations would silently corrupt the representation; explicit failures keep the output semantically valid.

3.3 Symbolic and Numeric Validation

A secondary validation layer evaluates all ln arguments over a user-supplied sample grid, reporting whether any sampled value is non-positive. The combined `validate_domain` check returns a structured report of symbolic and numeric safety, enabling analysis of expressions that the symbolic checker rejects conservatively.

4 Implementation

The transformation system is distributed as the Python package `eml` under `src/eml/`. The package separates transformation logic (`transform.py`), domain validation (`validation.py`), structural metrics (`core.py`), and experiment configuration (`config.py`), so that each concern can be verified independently.

4.1 Structural Metrics

Six metrics quantify the representational shift for each expression:

Metric	Definition
Nonlinear node count	Nodes of type exp, ln, or eml
Total node count	Size of the full SymPy expression tree
Tree depth	Maximum depth of the expression tree
EML primitive count	Number of eml$(\cdot,\cdot)$ nodes
Unique subexpressions	Distinct subexpressions by structural equality
Weighted circuit cost	Operator-weighted sum (EML/Pow: 3, exp/log/Mul: 2, Add/Abs: 1)

The weighted circuit cost assigns higher weight to operators that are more expensive to evaluate numerically, providing a rough proxy for computational cost under a target-hardware model.

4.2 The Schrödinger Operator

As a non-trivial test case, we apply the transformation to the 1D time-independent Schrödinger operator

$$H[\psi] = -\psi'' + V \cdot \psi, \tag{9}$$

with $\psi(x) = e^{-x^2}$ and $V(x) = x^2$. Derivatives are expanded symbolically before transformation. This produces a composite expression involving integer powers of x and nested exponentials—a realistic instance of the kind of expression arising in scientific computing.

4.3 Engineering Quality

The codebase is statically typed (PEP 484, verified by mypy with zero errors across 8 source files), formatted with Black (100-column limit), linted by Ruff, and import-sorted by isort. A pre-commit configuration enforces all checks at commit time. The test suite comprises 68 unit tests covering all public interfaces.

5 Experiments

All experiments are framed as representation analyses. None is designed to demonstrate performance superiority over either grammar.

5.1 Benchmark Suite

The benchmark comprises 23 expressions spanning polynomial, exponential, logarithmic, rational, and mixed forms (Table 1). Each expression is paired with a numeric validation range chosen to cover domain-safe and boundary cases.

Name	Expression
`cubic`	$x^3 + 2x - 1$
`square`	x^2
`exp_sum`	$e^x + e^{-x}$
`log_sum`	$\ln(x+3) + \ln(x+1)$
`x_exp_x`	$x\,e^x$
`gaussian`	e^{-x^2}
`sqrt_via_pow`	$(x^2+1)^{1/2}$
`log_abs`	$\ln\lvert x\rvert$
...	(23 expressions total)

Table 1: Representative benchmark expressions. The full suite spans polynomial, exponential, logarithmic, rational, and mixed forms.

5.2 Numeric Correctness Validation

For each domain-safe (expression, range) pair, we evaluate the original expression and its EML-evaluated form over 301 uniformly-spaced points, computing mean absolute error (MAE) and maximum absolute error (MaxAE). Expressions for which the original is not symbolically or numerically safe on the given range are excluded from this comparison.

5.3 Structural Complexity Comparison

EML and exp/log forms are compared on all six metrics defined above. Figure 1 illustrates the nonlinear node count and unique subexpression count across the benchmark.

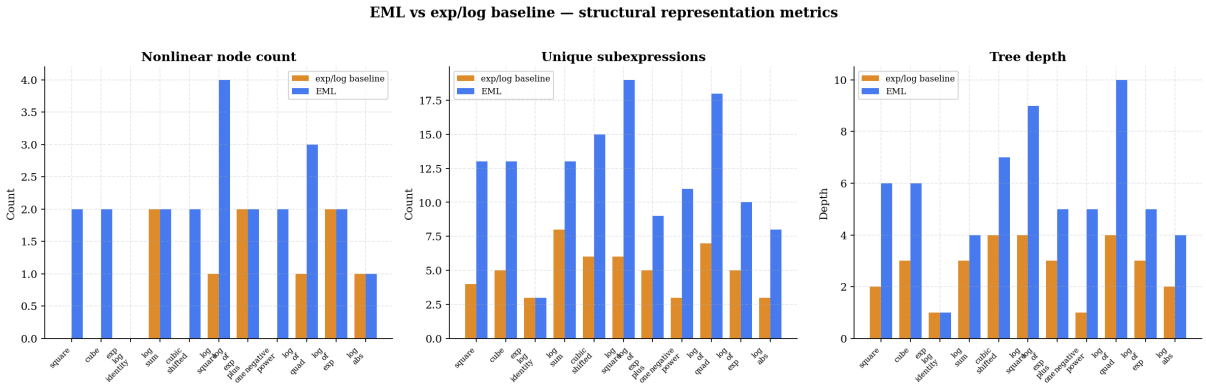

Figure 1: Structural representation metrics for EML vs standard exp/log rewrites across the 11 benchmark expressions that admit a domain-safe EML transformation. *Left:* nonlinear node count (exp/ln/eml nodes). *Centre:* unique subexpressions by structural equality. *Right:* total tree depth. EML encodings consistently introduce more nonlinear nodes and greater depth than the exp/log baseline; unique subexpression counts are subject to SymPy's automatic simplification. Expressions that raised a `DomainError` are excluded (see Section 8).

5.4 Evolutionary Symbolic Regression (Exploratory)

An elite-retention evolutionary search is applied to five target functions (Table 2), comparing the EML grammar against a standard SymPy exp/log grammar. Search parameters are held fixed across both grammars: 4 000 fitness evaluations, population of 100, elite retention of 15, maximum tree depth of 4, random seed 42. Final MSE and tree size are recorded for each grammar–target pair.

Target	Range
$x^2 + x$	$(0.1, 3.0)$
e^x	$(-2.0, 2.0)$
$\ln x$	$(0.1, 3.0)$
$x^2 \ln x$	$(0.1, 3.0)$
$e^x + \ln x$	$(0.1, 3.0)$

Table 2: Symbolic regression target functions and search ranges.

Table 3 summarises the final best-found MSE and tree size for each grammar–target pair. Figure 2 visualises the same data.

Target	EML MSE	EML nodes	Baseline MSE	Baseline nodes
$x^2 + x$	2.08×10^{-1}	39	6.20×10^{-1}	5
e^x	0.00×10^{0}	3	7.56×10^{-1}	5
$\ln x$	0.00×10^{0}	36	2.36×10^{-2}	5
$x^2 \ln x$	0.00×10^{0}	49	3.96×10^{-1}	5
$e^x + \ln x$	0.00×10^{0}	18	1.22×10^{0}	8

Table 3: Symbolic regression results: best-found MSE and tree size after 4 000 evaluations, population 100, elite 15, max depth 4, seed 42. MSE of 0 indicates exact recovery of the target within floating-point precision.

This experiment is *exploratory*. The evolutionary strategy is a minimal baseline; results characterise search-space structure under each grammar, not the capability of either grammar or any optimisation algorithm in general.

6 Results and Interpretation

Numeric correctness. Across all domain-safe (expression, range) pairs, the EML-evaluated form matches the original to within floating-point precision: mean absolute errors are at or below 10^{-10} for all passing cases. This confirms that the transformation is semantically faithful over its supported fragment—the representational changes are purely structural, not approximative.

Node inflation as representation overhead. The EML transformation consistently increases nonlinear node count relative to the exp/log baseline. This is a direct structural consequence of the encoding: a single exp becomes $\mathrm{eml}(\cdot, 1)$, and a single ln becomes $1 - \mathrm{eml}(0, \cdot)$, so the primitive count grows with every transcendental operation. This suggests that while EML reduces operator *variety*, it inflates operator *count*—a representational overhead that any downstream consumer of the tree must absorb.

Structural homogeneity and its implications. Despite the overhead, the EML encoding achieves a meaningful structural property: all nonlinear operations are expressed through a single operator type. Every exp and ln sub-expression is absorbed into a uniform layer of $\mathrm{eml}(\cdot, \cdot)$ nodes. This homogeneity is not merely aesthetic. In hardware compilation or algebraic circuit analysis, operator uniformity simplifies reasoning about the nonlinear layer—at the cost of the larger tree that must be traversed. The residual heterogeneity introduced by addition, which cannot be absorbed into the primitive, limits uniformity to the nonlinear sublayer rather than the full expression tree.

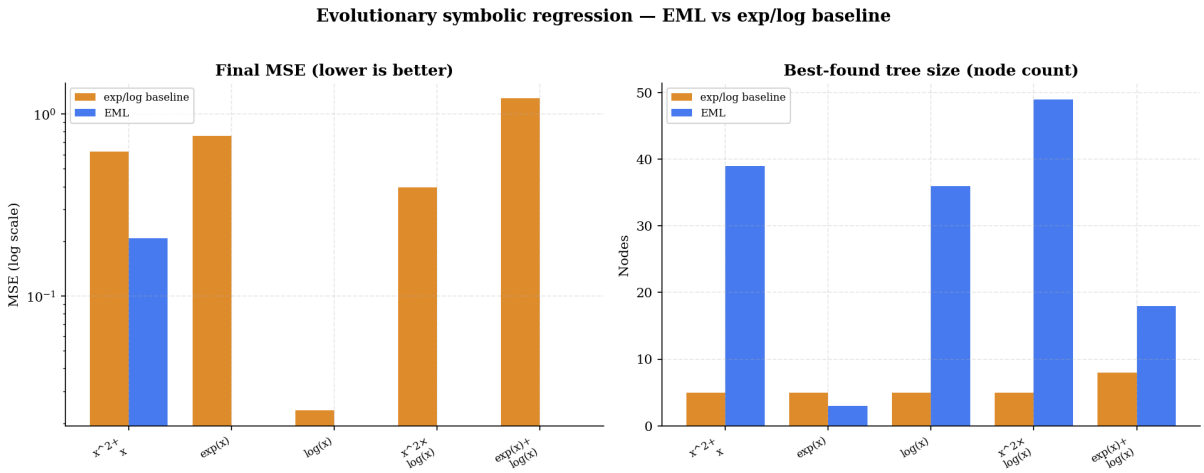

Figure 2: Evolutionary symbolic regression results across five target functions. *Left:* final best-found MSE on a log scale (lower is better). *Right:* best-found tree size (node count). The EML grammar recovers four of five targets exactly; the standard baseline produces compact but less accurate solutions. The $x^2 + x$ target is the exception: EML converges to a non-zero residual while the baseline produces a small linear approximation, reflecting the domain restriction that prevents multiplication absorption for unbounded x.

Depth increase and search-space consequences. EML encodings are consistently deeper than their exp/log counterparts. In the symbolic regression experiment, the EML grammar recovered four of five targets exactly (MSE $= 0$), while the standard baseline found only approximate solutions (Table 3). This apparent advantage, however, must be contextualised: exact EML recovery of $\ln(x)$ requires discovering the sub-tree $1 - \mathrm{eml}(0, x)$—a 36-node structure—while the baseline's best approximation used only 5 nodes. The EML grammar trades compactness for semantic precision. The single EML failure ($x^2 + x$, MSE ≈ 0.21) arises directly from the domain constraint: multiplication of unbounded x cannot be absorbed into the primitive, preventing construction of the quadratic term within the EML grammar. This confirms that representational coverage is the binding constraint, not search capacity.

Remark 2. *No result in this paper supports the claim that the EML grammar is superior or inferior to the exp/log baseline for any specific application. The experiments characterise structural and behavioural differences; their downstream relevance depends on the target system.*

7 Discussion

Uniformity as a representational trade. The central finding of this study is that EML-induced representations exhibit a consistent structural signature: fewer operator types, more operators, and greater depth. This is the representational price of operator uniformity. Whether the price is worth paying depends on the downstream computation. For hardware circuits or automatic differentiation frameworks that benefit from a homogeneous nonlinear basis, the uniform eml layer may simplify implementation at tolerable cost. For search-based methods where depth is a binding constraint, the overhead is likely detrimental.

Domain constraints as structural shapers. A recurring theme in this analysis is that domain constraints are not peripheral to the EML representation—they are constitutive of it. The strict positivity requirements for logarithm arguments determine which expressions can be absorbed into the primitive and which cannot. Expressions that are numerically safe on a restricted domain but involve symbolically unconstrained variables fail the rewrite entirely. This

reveals a deeper issue: the EML representation is not just a syntactic transformation but a *semantically typed* one. The type in question is positivity, and enforcing it at transform time is what prevents the representation from generating numerically invalid outputs. The trade-off between structural completeness and semantic safety is unavoidable in the real-valued setting.

Representation and search topology. Symbolic regression is, at its core, a search over a space of representations. The grammar determines the topology of that space—which expressions are adjacent, which are reachable in few steps, and which require long derivations. Replacing the standard exp/log grammar with the EML grammar changes the metric structure of this space: expressions that were shallow become deep, adjacencies shift, and the distribution of semantically meaningful trees changes. These effects are not unique to EML; they arise whenever the primitive set of a grammar is altered. What this study demonstrates is that such changes are measurable and non-trivial even for a restricted two-operator substitution.

Relation to the original theory. Odrzywołek's universality result [1] operates over the complex field with no computational constraints. The restrictions studied here—positive arguments, real outputs, no complex constants—constitute a strict subset of the conditions under which the theory applies. The structural differences we observe, particularly the incompleteness with respect to addition, are not contradictions of the theory; they are the expected consequences of restricting to a real-valued positive-domain fragment. The theory remains intact; this study characterises one slice of the space between the abstract operator and its practical instantiations.

8 Limitations

The following limitations are intrinsic to the scope and design of this study. They are reported explicitly to bound the conclusions that can be drawn.

1. **Restricted domain.** All ln arguments and non-integer Pow bases must be strictly positive. Expressions involving unconstrained symbols fail symbolic checks even when they are numerically safe on a restricted range.

2. **Addition is external.** The EML primitive does not absorb addition. Additive terms remain as standard `Add` nodes, so the full uniform-tree structure of the theoretical system is not realised.

3. **Incomplete function coverage.** Trigonometric, hyperbolic, and all non-exp-log elementary functions are unsupported and raise `UnsupportedExpressionError`.

4. **No complex domain.** The implementation is strictly real-valued. Complex-field identities that underpin several encodings in the original theory are unavailable.

5. **Conservative positivity checking.** SymPy's assumption system may reject expressions that are provably safe, requiring numeric fallback or manual domain annotation.

6. **Exploratory symbolic regression.** The evolutionary search is a simple baseline, not a competitive algorithm. Results describe search-space structure, not the optimisation capability of either grammar in general.

None of these limitations contradict the theoretical result of [1]. They reflect the additional constraints that practical real-valued computation imposes on an operator defined over the complex field.

9 Conclusion

We have studied how enforcing a single nonlinear primitive—the EML operator $\mathrm{eml}(x, y) = \exp(x) - \ln(y)$—changes the symbolic structure of expressions under real-domain constraints. The answer is precise: the transformation homogenises the nonlinear layer of expression trees at the cost of increased node count and tree depth, while domain constraints further restrict the fragment over which the transformation is applicable.

These structural changes carry concrete computational implications. For systems that benefit from operator uniformity, the EML representation may be an attractive intermediate form. For search-based methods that treat tree depth as a resource, the representational overhead is a measurable cost that must be budgeted for. In neither case does the choice of representation come for free.

The broader lesson is methodological: theoretical operator primitives do not translate transparently into practical representations. The gap between a universal complex-domain operator and its real-valued constrained instantiation is not merely an engineering inconvenience—it is structurally significant, and characterising it is a necessary step toward principled use of such primitives in applied symbolic computation.

Future work should investigate domain-aware rewriting via interval arithmetic, formal characterisation of the closure properties of the supported fragment, and gradient-based optimisation of EML circuits as studied in the original paper.

Acknowledgements

The EML operator and its universality theorem are due to Odrzywołek [1]. This work is an independent implementation study conducted to characterise one constrained instantiation of that result. All theoretical claims belong to the original work.

References

[1] Andrzej Odrzywołek. All elementary functions from a single binary operator. *arXiv preprint*, arXiv:2603.21852, 2026. https://arxiv.org/abs/2603.21852. doi:10.48550/arXiv.2603.21852.